The Pandas Workbench

A Visual Syntax Guide to the Data Wrangling Lifecycle



Raw Data

Wrangling / Transformation

Clean Data Table

```
>_

pip install pandas

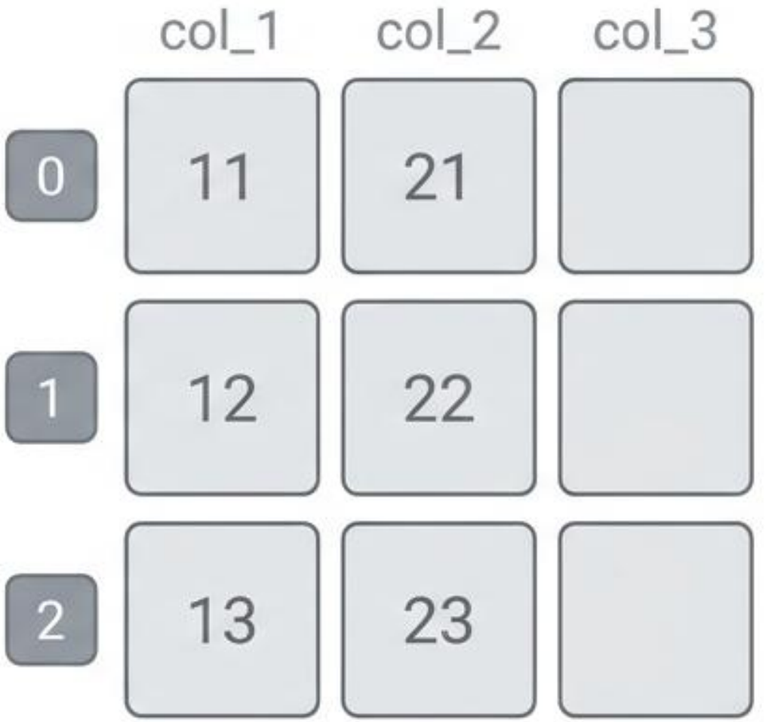
import pandas as pd
```

Gemini Notebook

Phase 1: Inception - Core Data Structures



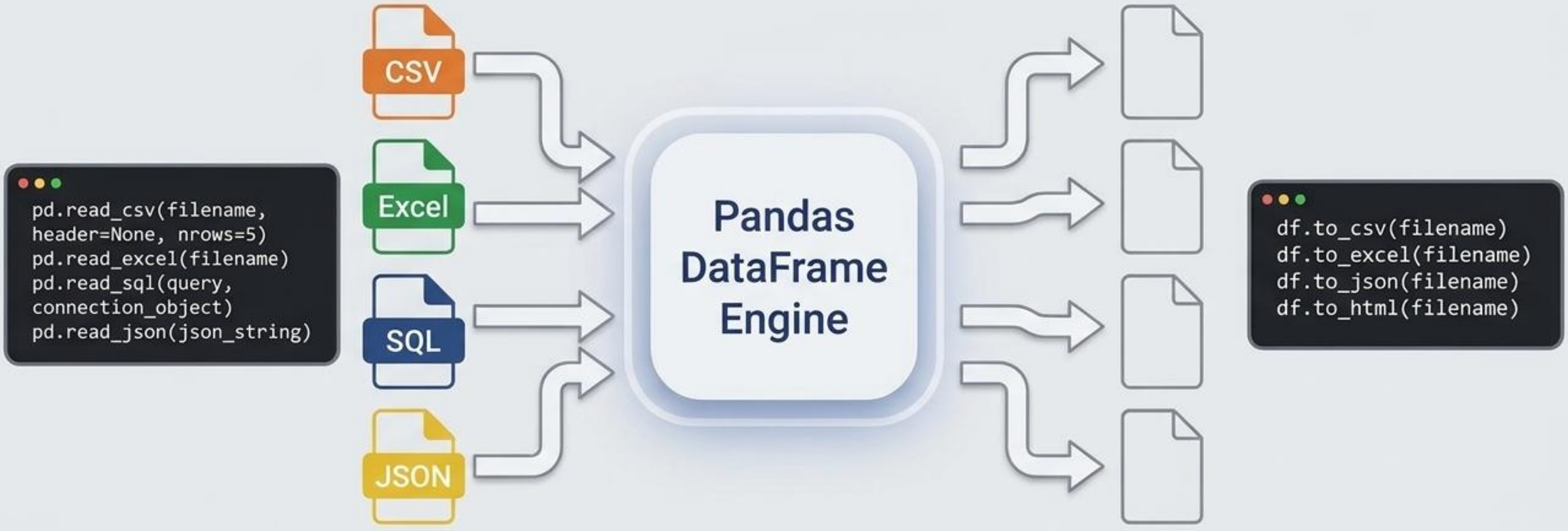

```
Series (1 Dimension)
s = pd.Series(['a', 'b', 'c'], index=[0, 1, 2])
```




```
DataFrame (Multidimensional)
df = pd.DataFrame({'col_1': [11, 12, 13],
'col_2': [21, 22, 23]}, index=[0, 1, 2])
```

Gemini Notebook

Phase 2: Ingestion & Output



Gemini Notebook

Phase 3: Diagnostics & System Monitor



The Overview

```
df.head(n) / df.tail(n)
    # First/Last n rows
df.sample(n)          # Randomly
    select n rows
df.shape              # Number of
    rows and columns
```



The Meta-Data

```
df.info()         # Index, Datatype,
    Memory info
df.describe()     # Summary stats for
    numericals
df.quantile([0.25, 0.75])# Quantiles
```

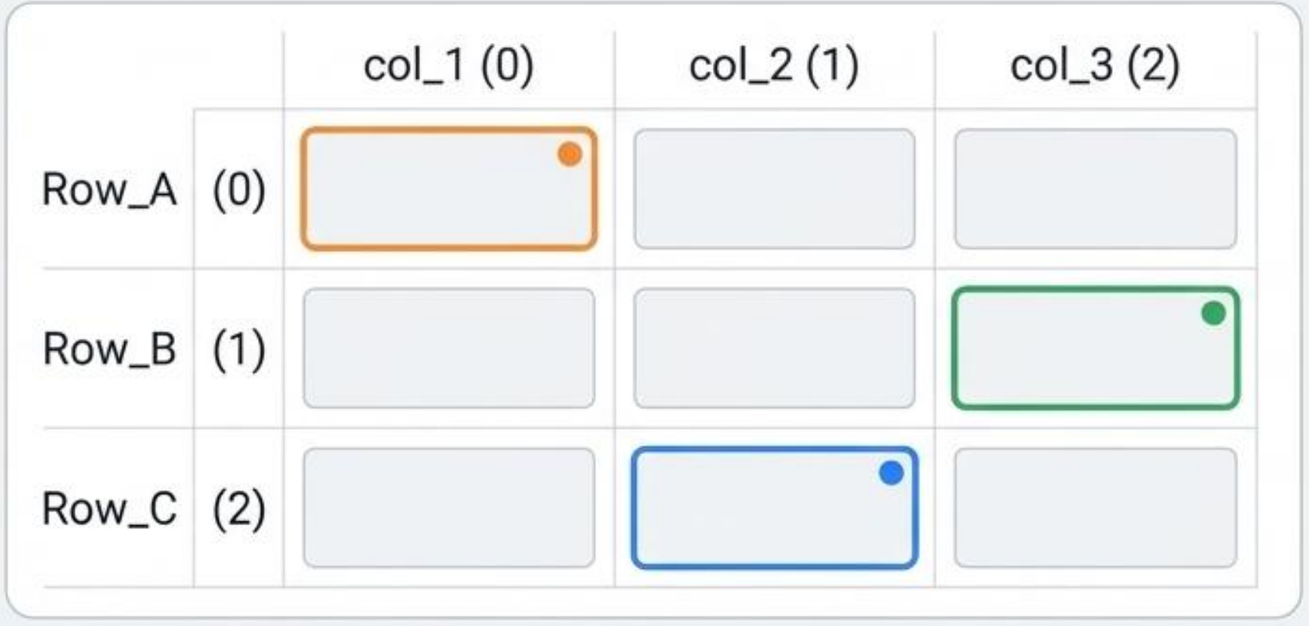


The Extremes

```
s.value_counts(dropna=False)
  # Unique values & counts
df.nlargest(n, 'col_1')
  # Select top n entries
df.nsmallest(n, 'col_1')
  # Select bottom n entries
```

Gemini Notebook

Phase 4: Surgery - The Anatomy of a DataFrame



Label-Based (The Names)

```
df.loc[df['col_1'] > 10, ['col_1', 'col_2']]
df.at['Row_C', 'col_2']  # Blue dot here
```

Position-Based (The Indices)

```
s.iloc[0] / df.iloc[0,:]
df.iloc[0,0]          # Orange dot here
df.iat[1, 2]          # Green dot here
```

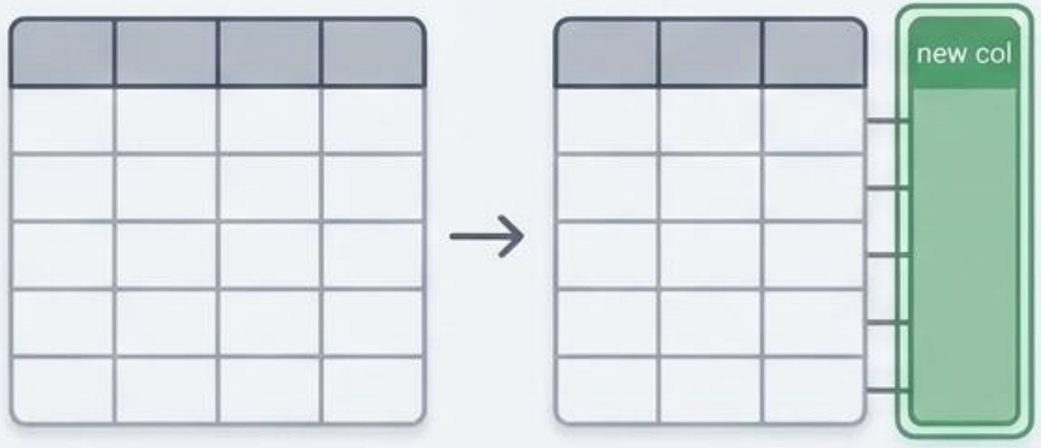
Column Quick-Grab

```
df['col_1']
df[['col_1', 'col_2']]
```

Gemini Notebook

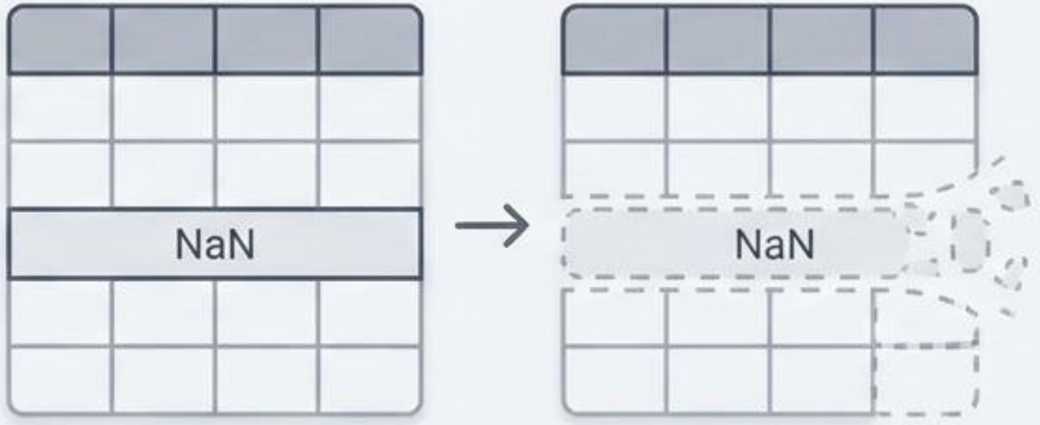
Phase 4: Surgery - Modification

Grafting (Adding)




```
df['new col'] = df['col'] * 100
df['new col'] = False
df.loc[-1] = [1, 2, 3]
```

Pruning (Dropping)




```
df.drop('col_1', axis=1)
s.drop([0, 1])
df.dropna()
df.dropna(axis=1, thresh=n)
```

Gemini Notebook

Phase 5: Transformation - Logical Filtering

	col_1 (int)	col_2 (str)	col_3 (str)
Row 1	120	a	X
Row 2	90	c	M
Row 3	150	a	Y
Row 4	200	b	Z
Row 5	10	a	N

```
Single Condition: df[df['col_1'] > 100]

Multiple Conditions (Boolean Logic): df[(df['col_1'] == 'a') & (df['col_2'] >= 10)]
```

Syntax Note: Use & for AND; | for OR. Notice the mandatory parentheses around each distinct condition.

Gemini Notebook

Phase 5: Transformation - Advanced Filtering Toolkit

Datetime Attributes

```
df[df['date'] > '2022-02-22']
df[df['date'].dt.month == 2]
```

Regex & Strings

```
df[df['col_1'].str.contains('pan*', regex=True)]
```

List Matching

```
df[df['col_1'].isin(['pan', 'das')]]
```

The Query API

```
df.query('col_1 > 100 and col_2 = 0')
```

Gemini Notebook

Phase 5: Transformation - Morphing Data

Casting & Morphing




```
df['col_1'].astype('int64') # or float, str
pd.to_numeric(df['col_1'], errors='coerce')
pd.to_datetime(df['date'], format='%Y-%m-%d')
df['col_1'].apply(pd.Series)
pd.DataFrame(df['Values'].tolist(), columns=['c1', 'c2'])
```

Order & Hierarchy

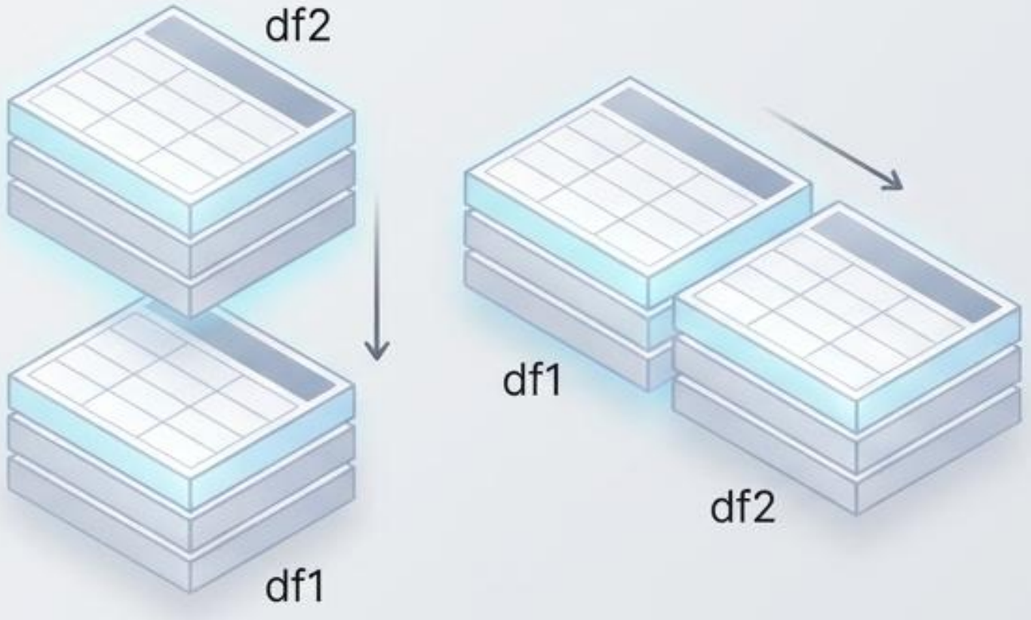



```
df.sort_values(by=['col_1', 'col_2'], ascending=False)
df.sort_index(ascending=False)
df.reset_index()
```

Gemini Notebook

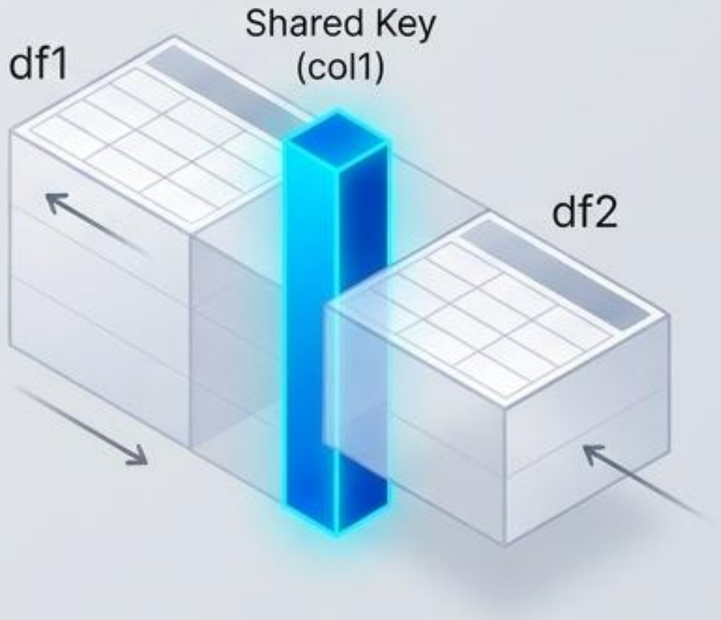
Phase 6: Synthesis - The Combination Matrix

Stacking (appending & Concatenating)




```
df1.append(df2)
pd.concat([df1, df2], axis=1)
```

Intersecting (Joining)

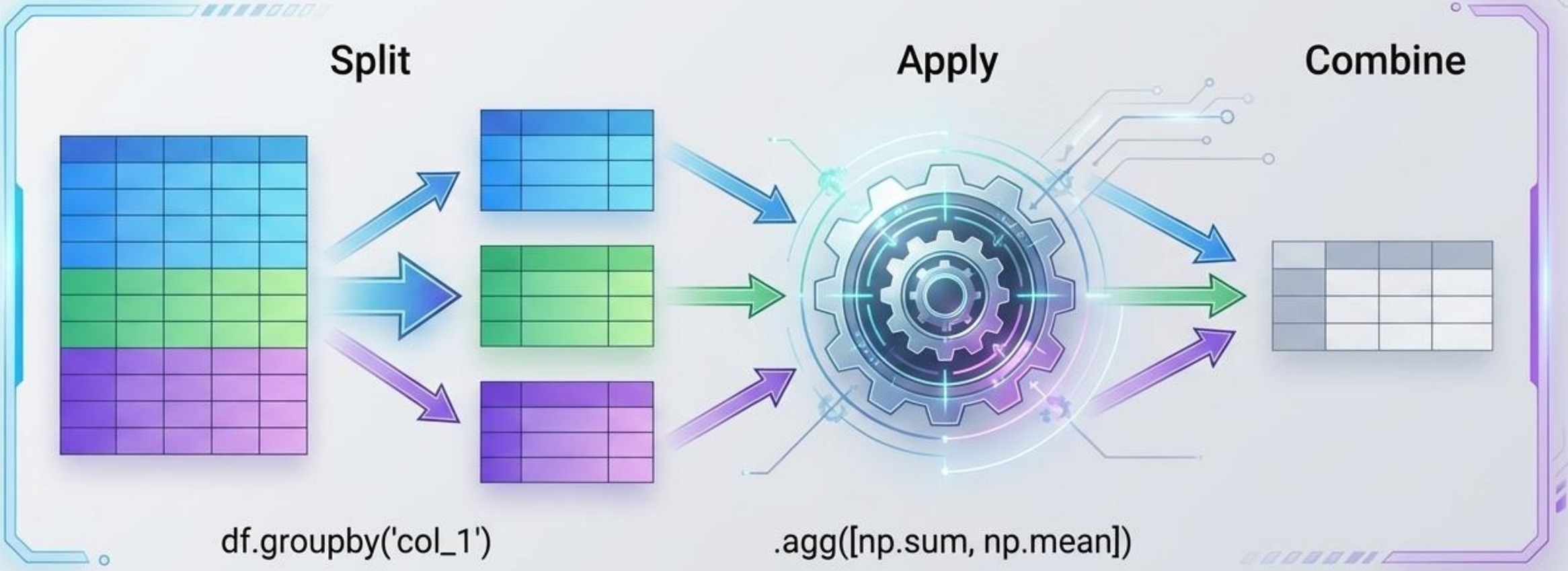



```
df1.join(df2, on='col1', how='inner')
```

Annotation: 'how' can be 'left', 'right', 'outer', 'inner'

Gemini Notebook

Phase 6: Synthesis - Groupby (Split-Apply-Combine)




```
df.groupby('col_1')
.groups
```

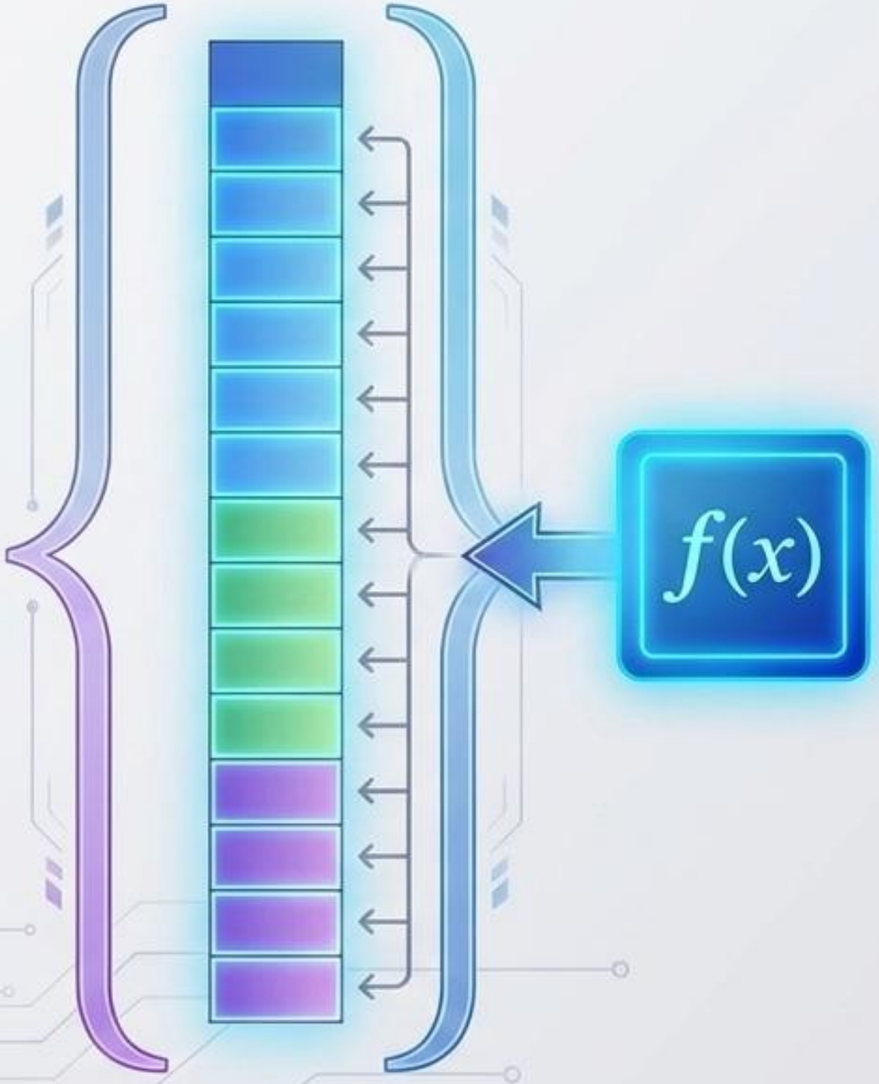
```
df.groupby('col_1')
.get_group(1)
```

```
df.groupby('col_1').filter
(lambda x: len(x) >= 5)
```

```
df.groupby('col_1')
.rank(method='dense')
```

Gemini Notebook

Phase 6: Synthesis - The Lambda Engine



Applying Standard Functions

```
def calc(x): return x + 1

df.apply(calc, axis=1) # Apply to rows
df[['col_1', 'col_2']].apply(calc) # Apply to columns
```

Applying Lambda Functions (Inline Custom Logic)

```
df.apply(lambda x: x * -1 if x < 0 else x)
```

Lambdas are perfect for quick, one-off conditional logic without formally defining a function block.

Gemini Notebook

Phase 6: Synthesis - Method Chaining

```
df = (pd.melt(df)
    .rename(columns={'variable': 'var'})
    .query('val >= 200')
    )
```

Why Chain?

Method chaining wraps operations in parentheses, eliminating intermediate variables, saving memory, and reading and reading like a logical sentence.

Gemini Notebook